

UNIDAD DIDÁCTICA 8

REALIZACIÓN DE PRUEBAS

**MÓDULO PROFESIONAL:
DESARROLLO DE INTERFACES**



CESUR
Tu Centro Oficial de FP

ÍNDICE

RESUMEN INTRODUCTORIO	2
INTRODUCCIÓN	2
CASO INTRODUCTORIO	3
1. REALIZACIÓN DE PRUEBAS	4
1.1 Validación y pruebas del desarrollo de una aplicación	5
1.2 Objetivo, importancia y limitaciones del proceso de prueba. Estrategias	6
1.2.1 Importancia y limitaciones	6
1.2.2 Estrategias	9
1.2.3 Metodologías	11
1.3 Versiones alfa y beta	12
1.4 Documentación de las pruebas	13
2. CLASIFICACIÓN DE LAS PRUEBAS	16
2.1 Pruebas unitarias o funcionales	17
2.2 Pruebas de integración: ascendentes y descendentes	18
2.3 Pruebas de capacidad y rendimiento	20
2.4 Pruebas de aceptación y usuario	22
2.5 Pruebas de sistema, seguridad y uso de recursos	23
2.6 Pruebas de regresión	25
2.7 Pruebas manuales y automáticas	26
2.8 Herramientas software para la realización de pruebas	27
2.8.1 NUnit	28
2.8.2 Sonarqube	29
2.8.3 JUnit	33
2.8.4 Bugzilla	34
2.8.5 XUnit	34
2.8.6 Apache JMeter	34
RESUMEN FINAL	40

RESUMEN INTRODUCTORIO

En esta unidad vamos a introducir y profundizar en los conceptos de la realización de pruebas de software.

Comenzaremos recordando las fases del desarrollo de software y en qué punto y momento las pruebas de software son importante, sus objetivos y las principales pruebas que hay que tener en cuenta para un desarrollo de una aplicación.

Dentro de las diferentes pruebas que disponemos, podemos realizar dos grandes clasificaciones, pruebas funcionales y pruebas no funcionales, o pruebas unitarias y pruebas de integración. En el primer caso, estudiaremos los diferentes tipos de pruebas que nos permiten comprobar aspectos individuales y funcionales de una aplicación, en el segundo testaremos la integración de los diferentes ámbitos de nuestro software.

Por último, mostraremos algunas aplicaciones disponibles en el mercado para la automatización de las pruebas, herramientas como JUnit que se integran perfectamente con nuestro IDE de desarrollo.

INTRODUCCIÓN

Cuando se está desarrollando un software, una de las tareas básicas del programador y de su equipo es hacer las pruebas necesarias para garantizar que el producto llega al cliente sin errores y cumpliendo los requisitos pedidos. Esta no es una tarea sencilla y se tiene que tener en cuenta desde el inicio del desarrollo. De hecho, un producto en continua evolución exige pruebas constantes y, por lo tanto, se convierte además en una de las etapas más costosas dentro del mantenimiento de la aplicación.

Debemos tener en cuenta que la programación, como actividad humana considerada muchas veces como “artesanal”, no está exenta de errores y, por lo tanto, las pruebas serán una gran ayuda para garantizar la calidad del producto final.

Cada elemento del software desarrollado hace falta que esté sometido a una prueba unitaria de forma individual para validar el funcionamiento de este elemento de software de forma independiente. A continuación, hay que llevar a cabo las pruebas de integración, de carga y de aceptación. Sin olvidar las pruebas de sistema y seguridad, así como las de regresión y las pruebas de humo.

CASO INTRODUCTORIO

Trabajas como desarrollador junior en una consultora de software de gestión empresarial, y trabajas con muchos y muy variados tipos de clientes. Ahora mismo estás centrado en una verticalización orientada a la gestión deportiva de clubs.

Quieres planificar y desarrollar un nuevo módulo para la aplicación en el que vas a almacenar la asistencia de los jugadores de equipos. Tu aplicación está desarrollada con Java, y en este desarrollo, además de planificar las especificaciones y los hitos de desarrollo, debes planificar las diferentes pruebas a realizar.

Al final de esta unidad, tendremos los conocimientos para poder conocer las diferentes pruebas que podemos realizar en una aplicación, la planificación de cuales realizar y las herramientas para llevarlas a cabo.

1. REALIZACIÓN DE PRUEBAS

Dentro del análisis de tu nuevo módulo debes planificar qué tipo de pruebas llevarás a cabo y cuándo las realizarás. Para ello planificarás todas las fases para la verificación y validación de tu módulo y, por último, planificarás unas pruebas alfa con personal indicado del club en las instalaciones y unas pruebas beta que realizará el personal de los clubes en su propio entorno.

Todas las fases establecidas en el desarrollo del software son importantes. La carencia o mala ejecución de alguna puede provocar que el resto del proyecto arrastre uno o varios errores que serán determinantes para el éxito del proyecto.



Ciclo de vida de desarrollo software

Fuente: <https://www.belatrixsf.com/blog/seguridad-desarrollo-software>

Una aplicación informática no puede llegar a manos de un usuario final con errores, y menos si esta es bastante visible y clara por haber sido detectada por los desarrolladores. Se daría una situación de carencia de profesionalidad y de confianza por parte de los usuarios en los desarrolladores. Esta pérdida de confianza evolucionaría en una carencia de futuras oportunidades.

Es por eso que esta fase del desarrollo de un proyecto de software se considera básica antes de hacer la transferencia del proyecto al usuario final. ¿Quién daría un coche por construido y finalizado si al intentar arrancarlo no funciona?

Hay que recordar las fases habituales en el desarrollo de un proyecto de software: toma de requisitos, análisis, diseño, desarrollo, pruebas, finalización y transferencia. Y también, es interesante resumir que los perfiles habituales de un equipo de trabajo en un proyecto de desarrollo de software son: jefe de proyecto, analista, diseñador, programadores y testadores. Con este entorno en mente nos hemos planteado diversas situaciones a lo largo del curso y ahora en concreto esta situación de pruebas test.



PARA SABER MÁS

Aquí tienes más información sobre el DevOps, que es la calidad del software:



1.1 Validación y pruebas del desarrollo de una aplicación

Todas las fases de desarrollo de un proyecto de software ofrecen la oportunidad de cometer errores por parte de todos los miembros de un equipo de trabajo. Algunos de estos errores serán más determinantes que otros y tendrán más o menos implicaciones en el futuro del desarrollo del proyecto.

En función de la importancia del error esta se podrá solucionar modificando algunas líneas de código o, quizás, se habrá trabajado en balde y habrá que volver a hacer todo el trabajo hecho.

Hay que remarcar dos conceptos antes de continuar: verificación y validación. Estos dos conceptos estarán directamente vinculados con las pruebas que verificarán formalmente una aplicación informática.

Estos dos conceptos, son el nombre que se da de manera genérica a los procesos que sirven para revisar que se asegura que el software desarrollado satisfará las especificaciones recogidas y que cubrirá las necesidades expresadas por el cliente. Hay que diferenciarlos de manera adecuada, aunque se acostumbran a confundir:

- La **validación** es la actividad encargada de asegurarse que la aplicación informática obtenida es la que se había determinado inicialmente y es la que se quería construir. La validación confirmará que la aplicación funciona tal como el cliente ha pedido. ¿El sistema hace el que el cliente quiera?
- La **verificación** es la actividad que comprueba el funcionamiento correcto del código programado. Se comprobará que tecnológicamente la aplicación informática se haya desarrollado de manera correcta. ¿El sistema hace el que tiene que hacer?

1.2 Objetivo, importancia y limitaciones del proceso de prueba. Estrategias

Los desarrolladores de software durante las fases del ciclo de vida del software pueden cometer innumerables errores humanos. Las pruebas nos permiten garantizar la calidad del software y representan una revisión final de las especificaciones, el diseño y la codificación. El objetivo de las pruebas es, por tanto, el proceso de ejecución de un programa en busca de errores.

Una definición más técnica de las pruebas sería: actividad en la cual un sistema o uno de sus componentes se ejecuta en circunstancias previamente especificadas siendo observados y analizados los resultados para evaluar determinados aspectos y determinar si cumple o no los requisitos especificados. Se puede decir que una prueba ha tenido éxito si descubre un error no detectado hasta ese momento.

Otro concepto relacionado con las pruebas es el de caso de prueba, que es un conjunto de entradas, condiciones de ejecución y resultados esperados desarrollados para un objetivo particular. Un buen caso de prueba es aquel que tiene grandes probabilidades de encontrar un error no descubierto.

Normalmente, una empresa de desarrollo de software emplea entre el 30 - 40% del esfuerzo total del proyecto en la realización de las pruebas.

Una de las limitaciones en la realización es que éstas no pueden asegurar la ausencia de errores, simplemente demuestran la existencia de estos.

1.2.1 Importancia y limitaciones

Algunas **afirmaciones** que debemos tener en cuenta y que están relacionadas con el término prueba son:

- El equipo de trabajo y los usuarios finales no tienen que participar en el proceso de prueba, sino que tienen que seleccionar un equipo externo al proyecto para el diseño y desarrollo.
- Una prueba tiene éxito si descubre un error no detectado hasta entonces. Un buen plan de prueba será aquel que tenga altas probabilidades de encontrar un error que no se haya descubierto hasta aquel momento.
- Hay que probar todos los módulos o partes de un proyecto de desarrollo del software (interfaces, documentación, ayuda...).
- Si en un módulo se encuentra un error, habrá una alta probabilidad de encontrar más en el mismo módulo o en otros módulos con una vinculación directa.

Como resumen, los expertos proponen una serie de **recomendaciones** sobre cómo se tienen que hacer las pruebas:

- No se tiene que ver el proceso de prueba como rutinario, sino como un proceso fundamental; por eso se tienen que destinar recursos, tiempos, personal experimentado y un proceso creativo.
- Los casos de prueba tienen que incluir tanto entradas correctas como incorrectas para evaluar el comportamiento del sistema en cualquier situación.
- Las pruebas se tienen que diseñar de forma que tengan la máxima probabilidad de encontrar el número más grande de errores con la mínima cantidad de esfuerzo y tiempo.
- El programador no tiene que probar sus propios programas. En las grandes empresas hay un equipo de prueba diferente del de desarrollo.
- Las pruebas se tienen que centrar e insistir más en las partes o módulos que más se utilizan o sean más críticos para el sistema.
- No se tiene que asociar el error a la negligencia de un programador; la finalidad de las pruebas tiene que ser encontrar errores y no desprestigiar nadie.

Por último, vamos a analizar las **limitaciones** de las pruebas:

- **Imposibilidad de asegurar la ausencia total de errores:** por más exhaustivas que sean, las pruebas no pueden garantizar que el software esté completamente libre de errores. Solo pueden demostrar la presencia de defectos detectados, no su ausencia total.
- **Dependencia de los casos de prueba:** la calidad y efectividad de las pruebas dependen en gran medida del diseño de los casos de prueba. Esto nos demuestra que hay que dedicar esfuerzo en el diseño de los casos de prueba.

- **Coste y tiempo elevado:** las pruebas representan un porcentaje significativo del esfuerzo total del proyecto, normalmente entre el 30% y el 40%. Sin embargo, errores detectados tardíamente pueden aumentar exponencialmente estos costos.
- **Errores humanos en el proceso de pruebas:** las pruebas manuales, en particular, son susceptibles a errores humanos, como omisiones o interpretaciones incorrectas de los resultados.



Coste corrección de un defecto dependiendo de la fase

Fuente: <https://www.mat.uson.mx/~mireles/gestionCalidad/GCcosto.html>



ENLACE DE INTERÉS

Accede y atiende a esta explicación del concepto de la detección de un defecto en software dependiendo de su fase.





VÍDEO DE INTERÉS

En este vídeo tienes una explicación más clara de la importancia del testing del software, especialmente desde el minuto 3.



1.2.2 Estrategias

La fase de pruebas verificará y validará que el producto final sea eficaz y efectivo. Para lo cual, hay que llevar a cabo el camino inverso al que se ha usado para el desarrollo del software.

Si se sigue el mismo razonamiento que se ha utilizado para construir, será más complicado encontrar los errores, puesto que el razonamiento será parecido y los errores cometidos se reproducirán.

Es por eso que hay que seguir un camino opuesto hay que ir del más pequeño (pruebas unitarias) al más grande (pruebas de integración), en vez del análisis top-down, que ha empezado con el análisis de los problemas más grandes para irlos dividiendo en problemas más pequeños.



Una de las metodologías de tests. TDD

Fuente: <https://ed.team/comunidad/que-es-el-tdd-61a0c8d7-d313-4dd6-bee2-e1696205aa36>

En las pruebas también hay que ir del más concreto (pruebas de subsistemas) al más general (pruebas de sistema).

La fase de pruebas se llevará a cabo justo antes de liberar un programa para la explotación (fase de finalización y transferencia) y hay que diferenciarla de la fase de mantenimiento, en la cual se llevan a cabo algunas modificaciones del programa, que habrá que probar también.

Una de las sorpresas que se suelen encontrar los jefes de proyecto, y también los programadores, es la enorme cantidad de tiempo y esfuerzo que requiere esta fase.



ENLACE DE INTERÉS

Conoce más sobre las estrategias del testing relacionado con SCRUM.



1.2.3 Metodologías

Dentro del proceso de pruebas se pueden emplear diferentes metodologías para diseñar y ejecutar los casos de prueba. Las dos principales metodologías son las pruebas de **caja negra** y las pruebas de **caja blanca**. Ambas se utilizan según las necesidades y el nivel de detalle requerido en el análisis del software.

Las **pruebas de caja negra** llevarán a cabo la comprobación del funcionamiento de un componente de software por medio de su interfaz, sin entrar a analizar su funcionamiento interno.

Conceptualmente se tratará de comprobar que el componente funciona correctamente cuándo:

- Se introducen datos correctos.
- Se introducen datos erróneos.
- Se introducen datos equivalentes.
- Se introducen valores extremos superiores e inferiores.



Diagrama caja negra

Fuente: <http://www.pmoinformatica.com/2016/04/pruebas-caja-negra-istqb.html>

Las **pruebas de caja blanca** son las encargadas de analizar la manera en que un componente de software resuelve un determinado problema atendiendo a los detalles internos de implementación.

Conceptualmente, se tratará de examinar el interior de un módulo de software desarrollado. Se verificará la corrección del siguiente:

- Sentencias, condiciones, bucles...
- Estructuración del código.
- Optimización del código.

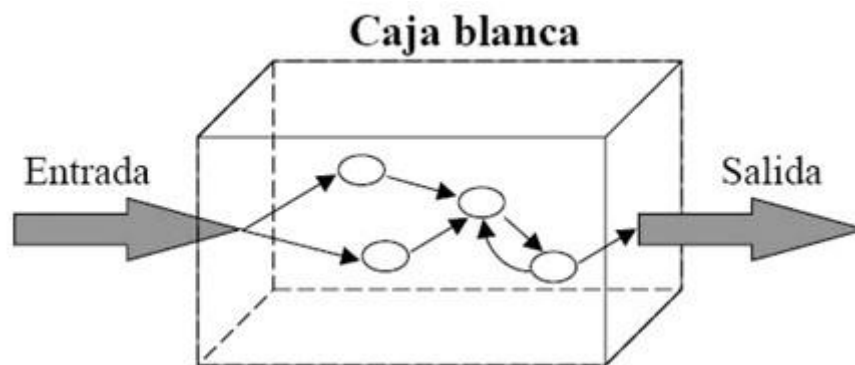


Diagrama caja blanca

Fuente: <http://visual-engin.com/2017/10/26/importancia-pruebas-de-software-testing/>

1.3 Versiones alfa y beta

La experiencia demuestra que, todavía después del más cuidadoso proceso de pruebas por parte del desarrollador y el equipo de trabajo, quedan una serie de errores que solo aparecen cuando el cliente pone en funcionamiento la aplicación o el sistema desarrollado.

Además, muchos desarrolladores ejercitan unas técnicas denominadas pruebas alfa (versión alfa) y pruebas beta (versión beta):

- Las **pruebas alfa** consisten en invitar al cliente que venga en el entorno de desarrollo a probar el sistema. Se trabaja en un entorno controlado y el cliente siempre tiene un experto a mano para ayudarlo a usar el sistema y para analizar los resultados.
- Las **pruebas beta** vienen después de las pruebas alfa, y se desarrollan en el entorno del cliente, un entorno que es fuera de control para el desarrollador y el equipo de trabajo. Aquí el cliente se queda a solas con el producto y trata de encontrar los errores, de los cuales informará el desarrollador.

Las pruebas alfa y beta son habituales en productos que se venderán a muchos clientes o que usarán muchos usuarios. Algunos de los compradores potenciales se prestan a estas pruebas bien para ir entrenando su personal con tiempo, bien en compensación de alguna ventaja económica (mejor precio sobre el producto acabado, derecho a mantenimiento gratuito, a nuevas versiones, etc.). La experiencia muestra que estas prácticas son muy eficaces.

En un entorno de desarrollo de software, tiene sentido cuando la aplicación o sistema para su desarrollo cuando lo use un gran número de usuarios finales (empresas grandes con diferentes departamentos que tendrán que utilizar esta nueva herramienta).

1.4 Documentación de las pruebas

La documentación de las pruebas es una parte más importante del proceso del desarrollo de software. La documentación consiste en describir en un **documento todo el proceso de pruebas**, desde su planificación hasta su ejecución, pero esta documentación estará muy ligada a la estrategia que hemos seguido para hacerlas por ello es muy importante previamente haber determinado una estrategia de pruebas.

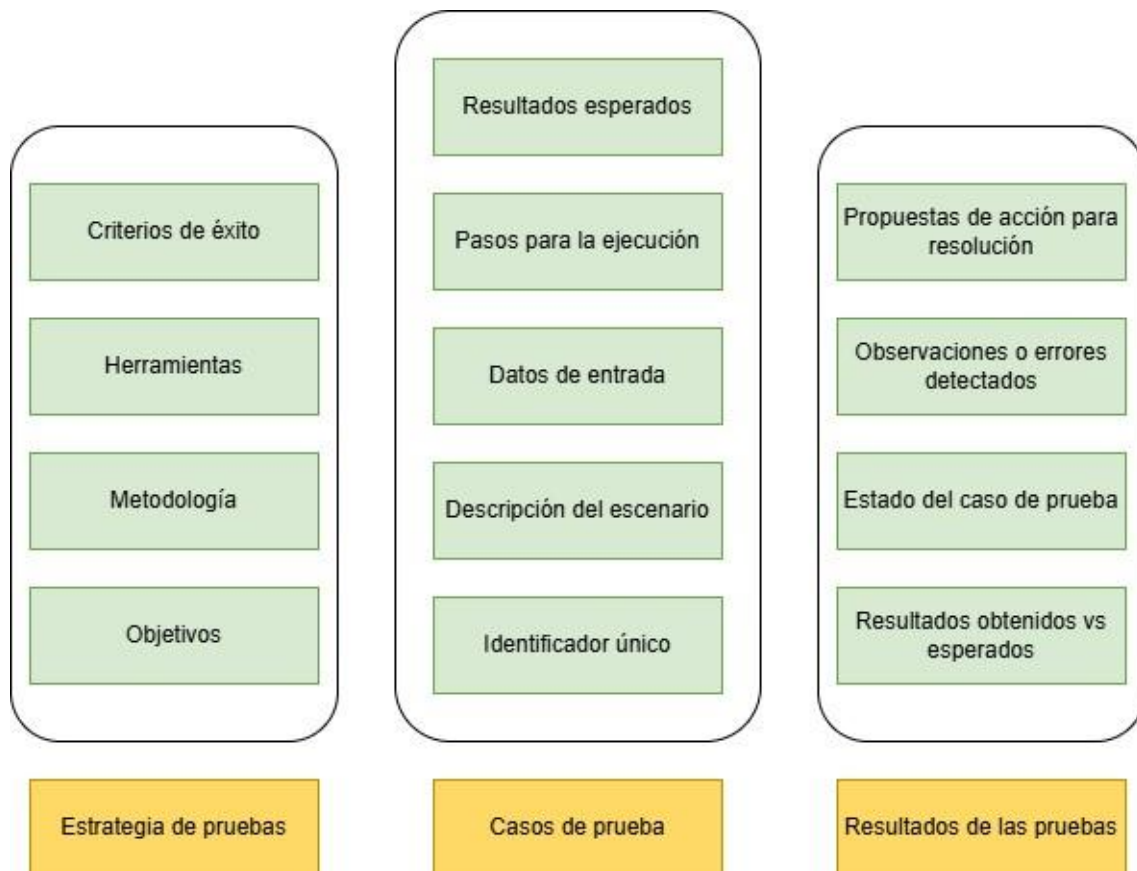
El propósito principal de documentar las pruebas es proporcionar un registro claro y estructurado del trabajo realizado, que permita:

- Asegurar que las pruebas se alineen con los requisitos definidos.
- Identificar errores y realizar un seguimiento de su resolución.
- Facilitar auditorías internas y externas.
- Servir como base para futuros ciclos de pruebas y desarrollos.

La documentación debe incluir al menos los siguientes elementos:

1. **Estrategia de pruebas:** consiste en describir las estrategias que vamos a seguir.
 - **Objetivos:** qué se evalúa (funcionalidad, rendimiento, seguridad, etc.).
 - **Metodología:** enfoques y técnicas utilizadas (caja negra, caja blanca, pruebas automatizadas, etc.).
 - **Herramientas:** software empleado (JUnit, JMeter, SonarQube, etc.).
 - **Criterios de éxito:** indicadores o métricas para determinar si una prueba es satisfactoria.
2. **Casos de prueba:** consiste en describir las pruebas que vamos a realizar (antes de realizarla):
 - Identificador único (e.g., TC001).
 - Descripción del escenario.
 - Datos de entrada.
 - Pasos para la ejecución.
 - Resultados esperados.
3. **Resultados de las pruebas:** consiste en documentar el resultado de las pruebas.
 - Resultados obtenidos versus los esperados.
 - Estado del caso de prueba (aprobado, fallido).

- Observaciones o errores detectados.
- Propuestas de acción para resolución.



Documentación de las Pruebas

Fuente: Elaboración Propia



ENLACE DE INTERÉS

Accede para encontrar una plantilla en formato Excel para definir tus casos de prueba.





EJEMPLO PRÁCTICO

El primer paso que nos propone nuestro jefe es encargarnos de la planificación de los tiempos e hitos de las pruebas a desarrollar dentro del desarrollo del módulo de asistencia de jugadores a los entrenamientos.

Teniendo en cuenta que en la empresa empleamos SCRUM como estrategia de planificación de los equipos y que tenemos los siguientes hitos:

- Sprint 1: desarrollo de las tablas para el almacenamiento de la asistencia de jugadores por equipo.
- Sprint 2: desarrollo del interfaz de entrenador para el control de asistencia.
- Sprint 3: desarrollo del dashboard de los coordinadores para el control de asistencia.

¿Cuál sería la planificación de las diferentes pruebas a realizar?

1. El primer paso es definir que pruebas pretendemos realizar:
 - a) Pruebas unitarias.
 - b) Pruebas de seguridad.
 - c) Pruebas de carga.
 - d) Pruebas de integración.
 - e) Pruebas de usabilidad.
 - f) Pruebas de aceptación.
2. El segundo paso es definir cuando se van a realizar todas esas pruebas. La primera acción es definir un Sprint 4 para la realización de las pruebas de integración.
 - a) **Pruebas unitarias.** Se realizarán a lo largo de todo el proyecto, de forma incremental y regresiva.
 - b) **Pruebas de seguridad.** Se realizarán tanto contra la base de datos como con el uso de la aplicación en el Sprint 4.
 - c) **Pruebas de integración.** Se realizarán en el Sprint 4.
 - d) **Pruebas de carga.** Se preparará una carga de datos y una simulación de usuarios de acuerdo a las especificaciones máximas de uso. Estas pruebas se realizarán en el Sprint 4.
 - e) **Pruebas de usabilidad.** Al estar trabajando con SCRUM, el usuario estará presente en todo el desarrollo, por lo que a partir del Sprint 2 se comenzarán a realizar las pruebas de usabilidad.
 - f) **Pruebas de aceptación.** Se realizarán al final del Sprint 4.
3. Por último, se planificarán los recursos y los equipos humanos que se necesitan para cada prueba:
 - a) Para las pruebas unitarias, el propio desarrollador definirá las mismas e incluso se puede utilizar la metodología TDD.
 - b) Para el resto de las pruebas se contará con un equipo independiente de definición y realización de pruebas. Al ser un módulo de un proyecto bastará con una única persona.

2. CLASIFICACIÓN DE LAS PRUEBAS

Teniendo ya planificadas las diferentes pruebas a realizar, comienzas con el desarrollo del módulo, y en este punto se puede comenzar a detallar de una forma más específica las pruebas.

Debes determinar los recursos que cada tipo de prueba va a requerir (herramientas, personal y tiempo). Partirás de las pruebas unitarias con Junit para hacer una integración ascendente y, una vez integrado los módulos, harás las pruebas de capacidad y aceptación. Por último, una vez estén integrados todos los módulos, harás las pruebas de sistema.

El proceso de las pruebas acostumbra a ser un procedimiento repetitivo y laborioso. Por eso se intentan automatizar algunos procesos de forma que los procedimientos de prueba se puedan hacer con la ayuda de programas informáticos.

Hay diferentes tipificaciones de las pruebas que se hacen a los proyectos de desarrollo de software.

En primer lugar, hay que tener en cuenta el momento para llevar a cabo las pruebas. Estas se pueden hacer en el momento de dar por finalizado un módulo de código de programación, o bien cuando todos los módulos están finalizados y se quieren integrar, o cuando la aplicación ya está finalizada e integrada y se quiere llevar a cabo una carga de datos reales, o bien si es el cliente quien hace las pruebas. Para estos casos se tienen los tipos de pruebas siguientes:

- Pruebas unitarias.
- Pruebas de integración.
- Pruebas de carga.

Otro tipo de clasificación se puede hacer en función de la manera de hacer las pruebas. En este caso, las pruebas pueden ser sin entrar en el detalle de la implementación, es decir, solo interactuando con las interfaces, o bien teniendo en cuenta los detalles internos de la implementación. En este caso, las pruebas se encuentran integradas dentro de las pruebas unitarias y se pueden clasificar en las siguientes:

- Pruebas unitarias.
- Pruebas de caja negra.
- Pruebas de caja blanca o transparente.
- Revisiones.

Una vez la aplicación desarrollada ya se encuentra validada, habrá que llevar a cabo otras pruebas, que se conocen como pruebas de sistema. Estas pruebas validarán la integración del software desarrollado en el sistema ya existente del usuario final o cliente. Algunas pruebas para desarrollar son:

- Rendimiento.
- Resistencia.
- Robustez.
- Seguridad.
- Usabilidad.
- Instalación.
- Pruebas de aceptación.

2.1 Pruebas unitarias o funcionales

Estas pruebas se definen a partir de funciones o características definidas en las especificaciones. La prueba unitaria o funcional se plantea a pequeña escala, y consiste a ir probando uno por uno (unidad por unidad) los diferentes módulos que constituyen una aplicación.

Las pruebas unitarias se definen como el procedimiento para probar el funcionamiento correcto de un módulo de código. Esto sirve para asegurar que cada uno de los módulos funcione correctamente por separado.

Conceptualmente, se trata de ir creando pruebas unitarias a medida que se vayan programando cada uno de los métodos/funciones o procedimientos de cada clase. Hay que validar el funcionamiento de un módulo antes de pasar al siguiente. Parece una tarea sencilla de implementar, pero a veces es difícil de llevar a cabo. En la práctica es difícil, puesto que el software es vivo y se tiene que ser muy sistemático porque cualquier variación en el código quede reflejada en la prueba unitaria correspondiente.



VÍDEO DE INTERÉS

Aquí tienes una introducción en el mundo de los test unitarios y las diferentes técnicas que nos encontramos.



2.2 Pruebas de integración: ascendentes y descendentes

En segundo lugar, figuran las pruebas software no funcionales que incluyen las pruebas de: rendimiento, carga, estrés, usabilidad, mantenibilidad, fiabilidad o portabilidad, entre otras. Por tanto, se centran en características del software que establecen “cómo trabaja el sistema”.

Las pruebas unitarias no pueden identificar todos los errores que tendrá el código por sí solas. Hay errores que pueden aparecer al integrar dos o más módulos o al cargar los datos o librar el sistema creado al usuario final. Es por esta razón que hay que tener en cuenta otros tipos de pruebas que prevean estos casos.

Las pruebas de integración tienen como base las pruebas unitarias y, consisten en una progresión ordenada de tests para los cuales los diferentes módulos van siendo ensamblados y probados hasta haber integrado el sistema completo. Consisten a verificar que un gran número de partes del software (módulos de la aplicación desarrollada) continúan funcionando cuando estos se han juntado.

A veces se produce la situación que unos componentes que funcionaban perfectamente por separado, al integrarse, fallan. Normalmente, se debe a un error en la comunicación entre unos y otros. Es conveniente que esta tarea la hagan personas ajenas en la programación de los módulos, tanto de los módulos independientes como de las personas que los han integrado. Si un programador que ha desarrollado un módulo detecta un error al integrarlo con otro y la soluciona sin dejar constancia, puede estar ocultando un error que afecte otras partes del código u otros módulos. Cuanto más tarde se detecta un error, más costosa será la resolución.



ENLACE DE INTERÉS

Conoce más sobre las diferentes pruebas para un desarrollo y en concreto sobre las pruebas de integración.



- **Prueba de integración ascendente**

Las pruebas de integración ascendentes empiezan combinando en grupos las unidades o módulos de bajo nivel para probarlos (pruebas dobles, triples, cuádruples, etc., según el número de módulos combinados).

Para hacer las pruebas, es necesario diseñar módulos software como, por ejemplo, componentes de pruebas, denominados programas controladores de pruebas o impulsores, que permiten simular el comportamiento de los módulos superiores.

Los programas controladores, tienen la misma interfaz del módulo que sustituyen, pero no hacen su función; por lo tanto, son fáciles de construir, puesto que no simulan la actividad de los módulos pare, sino que sirven simplemente para poder ejecutar el módulo que se probará.

- **Prueba de integración descendente**

En primer lugar, se prueban los módulos de nivel superior y después se van integrando los componentes de la capa siguiente. Una vez probados los componentes de esta capa se prueban los de nivel inferior y así sucesivamente hasta involucrar todas las capas.

Para poder hacer las pruebas descendentes sin incluir los módulos inferiores, necesitamos también componentes; concretamente, es necesario crear módulos software simuladores de pruebas denominados ficticios, que emulen el comportamiento y tengan la misma interfaz.

2.3 Pruebas de capacidad y rendimiento

El paso siguiente, una vez hechas las pruebas unitarias y las pruebas de integración, será llevar a cabo primero las pruebas de carga. Las pruebas de carga son pruebas que tienen como objetivo comprobar el rendimiento y la integridad de la aplicación ya acabada con datos reales. Se trata de simular el entorno de explotación de la aplicación.

Con las anteriores pruebas, (unitarias y de integración) quedaría probada la aplicación a escala de “laboratorio”. Pero también se necesita comprobar la respuesta de la aplicación en situaciones reales, e incluso, en situaciones de sobrecarga, tanto a escala de rendimiento como de descontrol de datos. Por ejemplo, una aplicación lenta puede ser poco operativa y no útil para el usuario.

Dentro de las pruebas de capacidad y rendimiento encontramos pruebas como las de volumen y estrés:

1. **Pruebas de volumen:** consiste en evaluar cómo el sistema maneja grandes cantidades de datos sin perder estabilidad ni integridad. Por ejemplo, podemos insertar millones de registros en la base de datos y así comprobar su tiempo de respuesta y el impacto en el sistema. Podemos realizar estas pruebas con herramientas como Apache JMeter y SQL Server Profiler.
2. **Pruebas de estrés:** consiste en probar el sistema más allá de sus límites de uso normal para observar cómo falla y cómo recupera su funcionalidad. Por ejemplo, podemos simular miles de usuarios concurrentes en un servidor web para medir el tiempo de respuesta y los errores generados. Podemos realizar estas pruebas con herramientas como Apache JMeter o LoadRunner.



VÍDEO DE INTERÉS

En este vídeo podrás ver cómo realizar pruebas de carga (capacidad) y estrés con JMeter. Puedes centrarte entre los minutos 11 y 20.





EJEMPLO PRÁCTICO

Nuestro jefe nos ha solicitado un diseño de integración de pruebas para el sistema de gestión de asistencia de jugadores a los entrenamientos con el fin de asegurar que los diferentes módulos de la aplicación funcionen correctamente al interactuar entre sí. ¿Cómo lo realizaríamos?

1. La estrategia debe validar que todos los módulos del sistema de gestión de asistencia, como la interfaz de usuario, la base de datos y el sistema de registro de asistencia, funcionen de manera conjunta. El objetivo es asegurar que los datos de los jugadores se registren correctamente en la base de datos y que la comunicación entre la interfaz de usuario y la base de datos sea fluida y sin errores.
2. Integración: se realizarán pruebas de integración ascendentes, comenzando con la integración de los módulos de más bajo nivel desde la base de datos y los módulos de validación de datos para posteriormente ir avanzando hacia los módulos de nivel superior, como la interfaz de usuario y los servicios web. Esta integración se hará progresivamente, verificando el comportamiento de cada conjunto de módulos a medida que se integran.
3. Herramientas: se utilizarán herramientas como JUnit para realizar las pruebas de integración en el código y asegurarse de que los módulos interactúan correctamente.
4. Criterios de éxito: La prueba será exitosa si:
 - a. Los módulos se comunican correctamente entre sí, sin errores de integración.
 - b. Los datos se transfieren correctamente entre la interfaz de usuario y la base de datos.
 - c. No se presentan errores de comunicación o de ejecución durante la integración de los módulos.

Vamos a definir un caso de prueba y un resultado para validar que el diseño se ha hecho de forma correcta.

1. Caso de prueba:

Identificador del Caso de Prueba	TC001
Descripción del Escenario	Registro de asistencia de un jugador.
Módulos a integrar	Módulo de Interfaz de Usuario, Módulo de Registro de Asistencia, Módulo de Base de Datos.
Datos de entrada	Nombre del jugador: "Juan Pérez", Equipo: "Equipo A", Asistencia: "Presente".
Pasos para la Ejecución	1. El entrenador accede a la interfaz de usuario.
	2. El entrenador selecciona el equipo "Equipo A" y el jugador "Juan Pérez".
	3. El entrenador marca la asistencia del jugador como "Presente".

	4. El sistema registra la asistencia en la base de datos.
	5. El entrenador verifica en la interfaz que la asistencia de "Juan Pérez" ha sido registrada correctamente.
Resultados Esperados	1. El sistema debe almacenar correctamente la asistencia en la base de datos con la fecha y hora del registro.
	2. La interfaz de usuario debe reflejar que el jugador "Juan Pérez" está marcado como "Presente".
Criterios de Éxito	Si la asistencia de "Juan Pérez" se almacena correctamente en la base de datos y se muestra correctamente en la interfaz de usuario, la prueba será exitosa.

2. Resultado de la prueba:

Paso de la prueba	Resultado Esperado	Resultado Obtenido	Estado de la Prueba
Paso 1: Acceso a la interfaz de usuario	El entrenador puede acceder correctamente al sistema.	El acceso a la interfaz es exitoso.	Aprobado
Paso 2: Selección del equipo y jugador	El entrenador selecciona correctamente el equipo "Equipo A" y el jugador "Juan Pérez".	El equipo "Equipo A" y el jugador "Juan Pérez" son correctamente seleccionados.	Aprobado
Paso 3: Marcado de la asistencia	El entrenador marca al jugador "Juan Pérez" como "Presente".	La asistencia de "Juan Pérez" se marca correctamente como "Presente".	Aprobado
Paso 4: Almacenamiento en la base de datos	El sistema debe almacenar la asistencia en la base de datos.	La asistencia se almacena correctamente en la base de datos.	Aprobado
Paso 5: Verificación en la interfaz	El entrenador ve la asistencia registrada en la interfaz de usuario.	La interfaz muestra correctamente que "Juan Pérez" está marcado como "Presente".	Aprobado

2.4 Pruebas de aceptación y usuario

Después de las pruebas de carga se encuentran las pruebas de aceptación, también denominadas de usuario. Estas pruebas las hace el cliente o usuario final de la aplicación desarrollada. Son básicamente pruebas funcionales, sobre el sistema completo, y buscan una cobertura de la especificación de requisitos y del manual del usuario. Estas pruebas, no se hacen durante el desarrollo, puesto que sería impresentable en orden al

cliente, sino una vez pasadas todas las pruebas anteriores por parte del desarrollador o el equipo de tests.

El objetivo de la prueba de aceptación es obtener la aprobación del cliente sobre la calidad del funcionamiento del sistema desarrollado y probado.

La experiencia demuestra que todavía, después del más cuidadoso proceso de pruebas por parte del desarrollador y el equipo de trabajo, quedan una serie de errores que solo aparecen cuando el cliente pone en funcionamiento la aplicación o el sistema desarrollado.



ENLACE DE INTERÉS

Comprueba varios ejemplos de plantillas para las pruebas:



2.5 Pruebas de sistema, seguridad y uso de recursos

Las pruebas de sistemas y seguridad son aquellas pruebas que se llevarán a cabo una vez finalizadas las pruebas unitarias (cada módulo por separado), las pruebas de integración de los módulos, las pruebas de carga y las pruebas de aceptación por parte del usuario.

Temporalmente se encuentran en una situación en la cual el usuario ha podido testar la aplicación desarrollada llevando a cabo las pruebas de aceptación. Posteriormente, la aplicación se ha integrado a su nuevo entorno de trabajo. Las pruebas de sistema servirán para validar la aplicación ya validada una vez ha sido integrada con el resto del sistema del usuario.

Algunos tipos de pruebas para desarrollar durante las pruebas del sistema son:

1. **Pruebas de rendimiento:** valorarán los tiempos de respuesta de la nueva aplicación, el espacio que ocupará en disco, el flujo de datos que generará a través de un canal de comunicación...
2. **Pruebas de resistencia:** valorará la resistencia de la aplicación para determinadas situaciones del sistema.
3. **Pruebas de robustez:** valorará la capacidad de la aplicación para soportar varias entradas no correctas.
4. **Pruebas de seguridad:** estas pruebas ayudarán a determinar los niveles de permisos de los usuarios, las operaciones que podrán llevar a cabo y las de acceso al sistema y a los datos.
5. **Pruebas de usabilidad:** estas pruebas determinarán la calidad de la experiencia de un usuario en la manera en la cual este interactúa con el sistema.
6. **Pruebas de instalación:** estas pruebas indicarán las operaciones de arranque y actualización de los softwares.

Las pruebas de sistema aglutinan tantas otras pruebas que tendrán varios objetivos:

- Observar si la aplicación hace las funciones que tiene que hacer y si el nuevo sistema se comporta como lo tendría que hacer.
- Observar los tiempos de respuesta para las diferentes pruebas de rendimiento, volumen y sobrecarga.
- Observar la disponibilidad de los datos en el momento de recuperación de un error.
- Observar la usabilidad.
- Observar la instalación (asistentes, operadores de arranque de la aplicación, actualizaciones del software...).
- Observar el entorno una vez la aplicación está funcionando a pleno rendimiento (comunicaciones, interacciones con otros sistemas...).
- Observar el funcionamiento de todo el sistema a partir de las pruebas globales hechas.
- Observar la seguridad (el control de acceso e intrusiones...).

Dentro de las pruebas que hemos visto, las de seguridad son muy importantes, ya que sirven para identificar vulnerabilidades y proteger los datos y operaciones del sistema. Para ello verifican que el sistema maneje adecuadamente las políticas de autenticación, autorización y encriptación de datos sensibles. Estas pruebas suelen incluir técnicas

como pruebas de penetración (pentesting), evaluación de políticas de acceso y análisis de configuración de seguridad. Para evaluarlas tenemos herramientas como SonarQube o OWASP.

Las pruebas de uso de recursos permiten verificar que una aplicación utiliza los recursos del sistema de manera eficiente y que no ocasionará problemas de rendimiento en entornos reales. Los recursos a examinar son la memoria RAM, CPU, almacenamiento y la red. Si no se hacen estas pruebas podemos tener problemas como fugas de memoria, excesivo consumo de CPU o saturación de ancho de banda. Herramientas como Apache JMeter o VisualVM ayudan a evaluar el uso de recursos y a identificar cuellos de botella en el sistema.

2.6 Pruebas de regresión

Las pruebas de regresión son un tipo más de pruebas de software. Estas pruebas de regresión buscan detectar posibles nuevos errores o problemas que puedan salir al haber introducido cambios o mejoras en el software.

Estos cambios pueden haber sido introducidos para solucionar algún problema detectado en la revisión del software a raíz de una prueba unitaria o de integración o de sistema. Además, pueden solucionar un problema, pero provocar otros, sin haberlo previsto, en otros lugares de los softwares. Es por esta razón que hay que llevar a cabo las pruebas de regresión al finalizar el resto de las pruebas.

Un control no conveniente de los cambios de versiones, o una falta de consideración hacia otros módulos o partes del software, pueden ser razones de los problemas para detectar en las pruebas de regresión.

Se puede automatizar la detección de este tipo de errores con la ayuda de herramientas específicas. La automatización es complementaria al resto de pruebas, pero facilitará la repetibilidad.

Guía de Pruebas de Regresión



Infografía pruebas de regresión

Fuente: <https://www.facebook.com/qatesterlatam/photos/a.1507566522685359/1794318550676820/>

2.7 Pruebas manuales y automáticas

Una de las sorpresas que se suelen encontrar los jefes de proyecto y, también, los programadores, es la enorme cantidad de tiempo y esfuerzo que requiere esta fase, el coste de la fase de pruebas puede fácilmente superar el 80% del coste total del proyecto. La fase de pruebas es una de las partes del proyecto que muchos programadores todavía consideran clasificable como arte, es decir, difícilmente medible, y no suelen valorar bastante esta fase, a pesar del enorme impacto que puede llegar a tener en el coste de desarrollo.

El número de pruebas no se puede valorar en función del número de líneas de código, de variables o de valores que se usen en el desarrollo del software. Las pruebas para ejecutar no tienen que ser necesariamente finitas. Al usar bucles en los cuales es fácil introducir condiciones para un funcionamiento sin un final, el número de alternativas y combinaciones posibles para desarrollar las pruebas pasa a ser exponencial, y se hace muy difícil (por no decir imposible) la identificación y ejecución completa a todos los efectos prácticos.

Existen dos formas de realizar la prueba de software:

- **Prueba manual:** la prueba manual se realiza ejecutando el software, introduciendo datos de entrada y evaluando los de salida. Este tipo de prueba cuenta con la ventaja de que se ejecutan las pruebas exactamente como el usuario final. Con la prueba manual, otros desarrolladores o los mismos usuarios finales no pueden verificar el correcto funcionamiento de nuestra aplicación ya

que deben aceptar que nuestra prueba ha sido satisfactoria o realizarla ellos mismos de nuevo. También presenta algunos inconvenientes:

- **Repetitiva.** Si se cambia o añade una funcionalidad nueva es necesario volver a probar las aplicaciones para asegurarnos de que no se ha producido un error por causa de este cambio.
- **Susceptible de error.** Precisamente por el hecho de ser repetitiva, es posible que pasemos por alto detalles a la hora de probar, o no se prueben funcionalidades aparentemente no relacionadas con el cambio, pero realmente afectadas.
- **Prueba automatizada:** automatizando las pruebas resolvemos muchos de los problemas planteados anteriormente:
 - **Elimina el componente repetitivo.** Se transmite el trabajo repetitivo al ordenador que seguirá exactamente la secuencia de pruebas, cada vez que lo solicitemos.
 - **Reduce los errores.** Cada repetición será exactamente igual cada vez que se ejecute, de forma que se puede probar fácilmente todas las funcionalidades ante cualquier cambio sin temor a olvidar alguna parte.
 - Permite **probar las partes no visibles de código.**
 - Permite a los usuarios finales u otros desarrolladores verificar **que el software funciona** como debe.

Pero esto no queda aquí. Estudios han demostrado que la automatización de pruebas es una de las mejores inversiones que se puede realizar ya que produce resultados muy satisfactorios además de que es la única actividad que garantiza la calidad del software.

El problema radica en que pocas veces es posible automatizar el 100% de las pruebas y es entonces cuando nos surge la pregunta de qué pruebas debemos hacer de forma manual y cuales automatizar.

2.8 Herramientas software para la realización de pruebas

Hay muchas herramientas que ayudarán al desarrollo de las pruebas. Algunas herramientas están integradas a los entornos de desarrollo integrados, pero otras herramientas son independientes y habrá que seleccionarlas en función del entorno y lenguaje de programación utilizado.

Algunos ejemplos de estas herramientas independientes pueden ser:

- Junit

- Bugzilla
- NUnit
- XUnit
- Sonarqube
- Apache JMeter



ENLACE DE INTERÉS

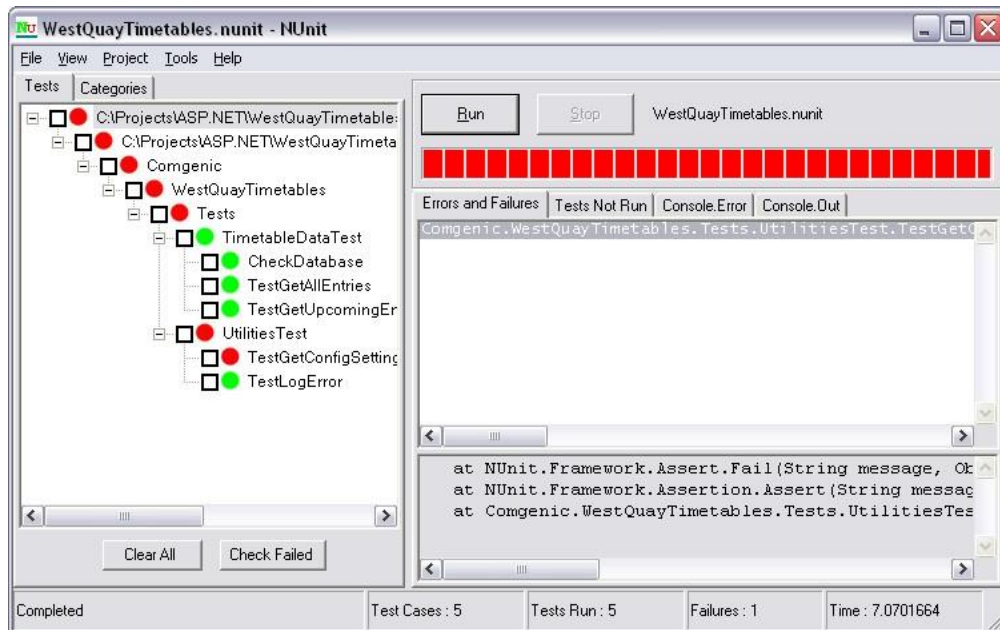
A modo de breve introducción, aquí puedes comprobar algunas de las herramientas mencionadas en este epígrafe:



2.8.1 NUnit

Es una evolución de la herramienta JUnit, pero que acepta otros muchos lenguajes. De hecho, es un entorno de trabajo para Microsoft .NET de forma que permitirá trabajar en todos los lenguajes .NET (Visual Basic, C#, ASP.NET, ...)

NUnit ofrece un nivel más bajo de integración con otras herramientas.



Ejemplo de ejecución

Fuente: ioc.xtec.cat

2.8.2 Sonarqube

Es una herramienta poderosísima para realizar análisis de código y de proyectos. Con esta herramienta, además de realizar análisis de errores, podremos realizar un análisis de la calidad de software, es decir nos realizará un análisis de código duplicado, código mejorable o código potencialmente peligroso desde el punto de vista de la seguridad.



Ejemplo de ejecución

Fuente: <https://www.sonarqube.org/>



ENLACE DE INTERÉS

Aquí podrás encontrar las librerías y documentación de Sonarqube.

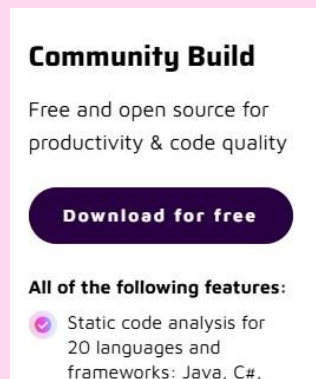


EJEMPLO PRÁCTICO

Estamos desarrollando un módulo de gestión de inventario para una aplicación empresarial utilizando Java y Maven. Nuestro jefe nos pide integrar pruebas con Sonarqube para garantizar la calidad del código.

¿Cuáles serían los pasos?

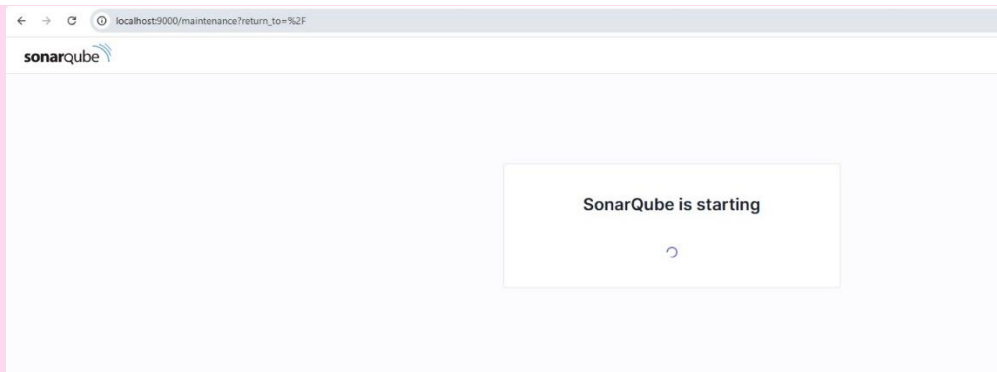
1. El primer paso es descargar la edición Community Edition de Sonarqube en su web oficial.



Descarga Sonarqube

Fuente: Elaboración Propia

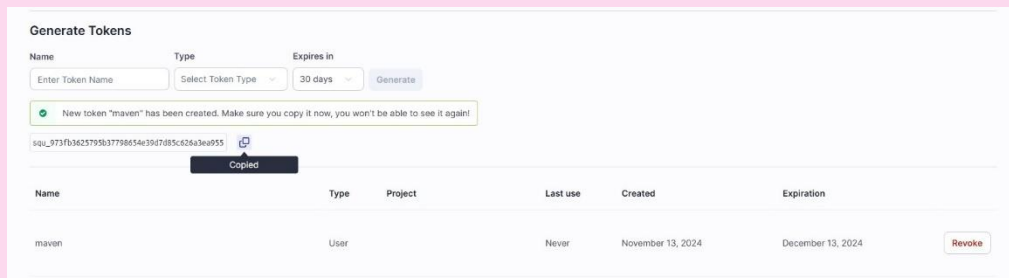
2. Descomprimir el archivo descargado en una carpeta, por ejemplo, C:\SonarQube, luego navega a C:\SonarQube\bin\windows-x86-64 (o la correspondiente a tu sistema operativo) y ejecuta el bat de Start. Ahora, si accedemos a <http://localhost:9000> deberemos ver esto. (Sonarqube no funciona con la versión 21 de java por lo que se recomienda utilizar versiones LTS como la 17)



Lanzamiento Sonarqube

Fuente: Elaboración Propia

- Introducimos las credenciales admin, admin y nos pedirá cambiar la contraseña, esta contraseña se puede cambiar a la que quieras, por ejemplo: MiContraseña1a123.
- Generaremos un token para autenticarnos: Mi cuenta < Seguridad < Tokens y metemos los datos, después guarda el token porque lo necesitaremos.



Generación Token Sonarqube

Fuente: Elaboración Propia

- Ahora abrimos o creamos el proyecto que se vaya a analizar. Dado que es importante que esté en un Maven, cogeremos cualquier proyecto creado durante el curso en Maven.
- Añadimos lo siguiente al pom.xml

```
<build>
  <plugins>
    <!-- Plugin de SonarQube -->
    <plugin>
      <groupId>org.sonarsource.scanner.maven</groupId>
      <artifactId>sonar-maven-plugin</artifactId>
      <version>3.9.1.2185</version>
    </plugin>
  </plugins>
</build>

<properties>
  <sonar.projectKey>TU_PROYECTO</sonar.projectKey>
  <sonar.host.url>http://localhost:9000</sonar.host.url>
  <sonar.login>TU_TOKEN</sonar.login>
</properties>
```



```
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <groupId>com.ecommercehibernate</groupId>
  <artifactId>EcommerceHibernate</artifactId>
  <version>0.0.1-SNAPSHOT</version>

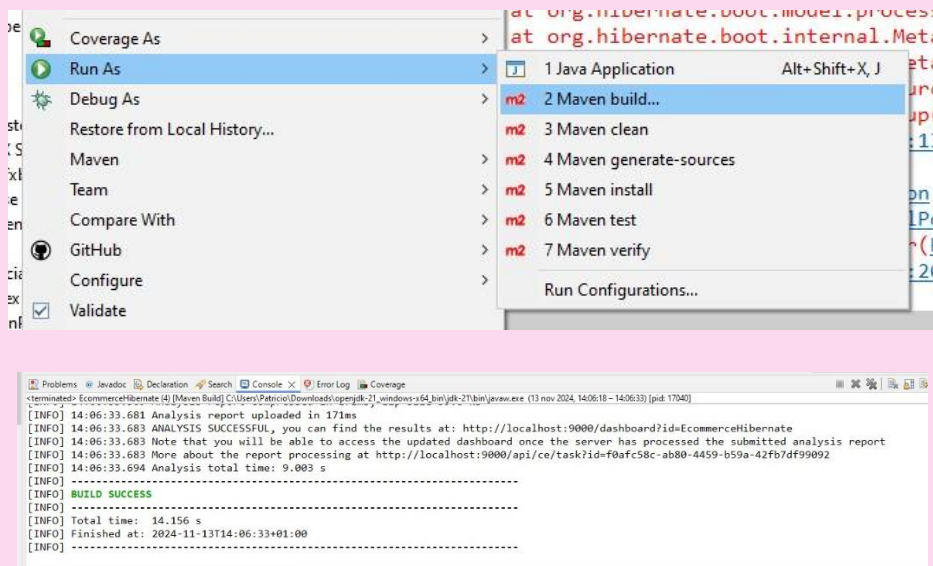
  <properties>
    <sonar.projectKey>EcommerceHibernate</sonar.projectKey>
    <sonar.host.url>http://localhost:9000</sonar.host.url>
    <sonar.login>squ_973fb3625795b37798654e39d7d85c626a3ea955</sonar.login> <!-- Sustituye YOUR_SONARQUBE_TOKEN por tu token -->
  </properties>

  <build>
    <plugins>
      <!-- Plugin de SonarQube -->
      <plugin>
        <groupId>org.sonarsource.scanner.maven</groupId>
        <artifactId>sonar-maven-plugin</artifactId>
        <version>3.9.1.2185</version> <!-- or latest stable version -->
      </plugin>
    </plugins>
  </build>
</project>
```

Ejemplo pom.xml Sonarqube

Fuente: Elaboración Propia

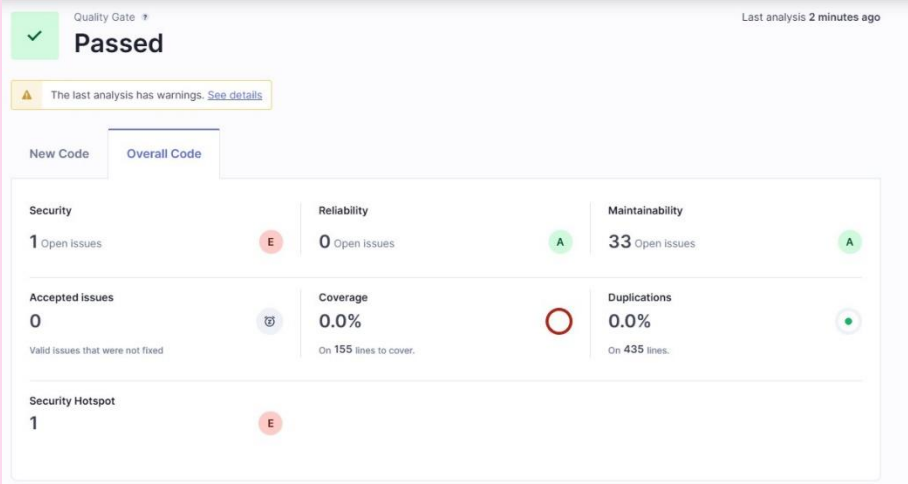
7. Ahora lo ejecutaremos como una “Maven Build” y en “goals” escribimos: sonar:sonar



Ejecución pruebas

Fuente: Elaboración Propia

8. Ahora si volvemos a <http://localhost:9000> podremos ver el análisis en detalle de nuestro proyecto.



Quality Gate * Last analysis 2 minutes ago

Passed

⚠ The last analysis has warnings. [See details](#)

New Code Overall Code

Security 1 Open issues E	Reliability 0 Open issues A	Maintainability 33 Open issues A
Accepted issues 0 0 Valid issues that were not fixed	Coverage 0.0% 0 On 155 lines to cover.	Duplications 0.0% 0 On 435 lines.
Security Hotspot 1 E		

Resultado pruebas
Fuente: Elaboración Propia

2.8.3 JUnit

Es una herramienta de pruebas para proyectos desarrollados en el lenguaje de programación Java. Esta herramienta sirve para llevar a cabo pruebas unitarias ejecutando clases Java de manera controlada para evaluar el comportamiento de cada método. Se trata de una herramienta de código abierto.

JUnit es una herramienta que también se usa para automatizar las pruebas de regresión. Es una herramienta que se puede usar de manera independiente o integrada en otras herramientas de desarrollo de Java, como lo Eclipse.



ENLACE DE INTERÉS

Accede a las librerías y documentación de JUnit.



2.8.4 Bugzilla

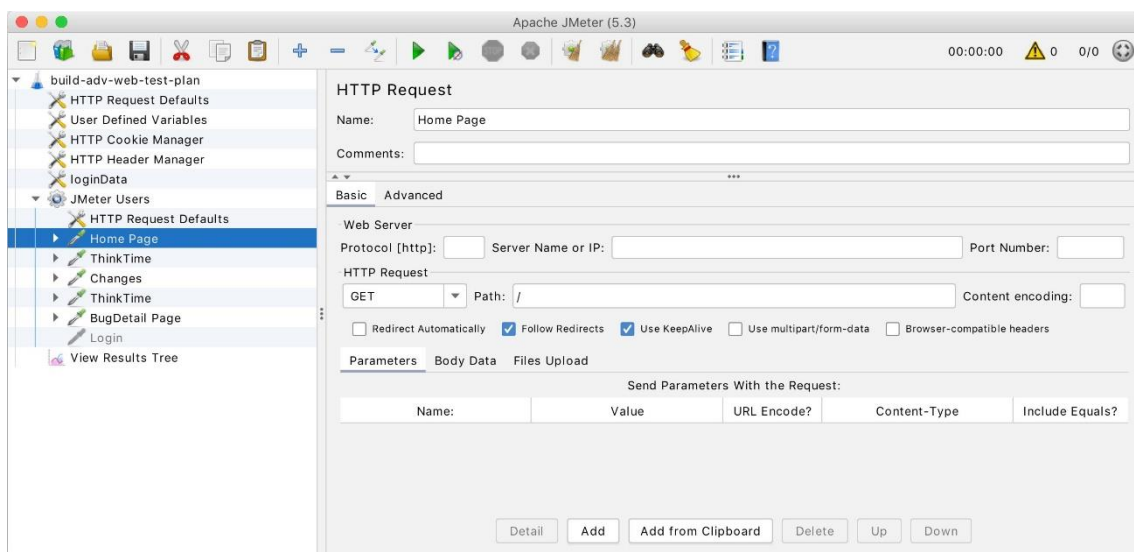
Bugzilla es una herramienta de seguimiento de errores y gestión de incidencias utilizada en proyectos de software. Permite a los equipos de desarrollo registrar, rastrear y gestionar problemas o solicitudes de mejora en tiempo real. Es código abierto y sirve para proyectos que requieren colaboración y un enfoque en la resolución de defectos.

2.8.5 XUnit

XUnit es una familia de marcos de pruebas unitarias que sigue el diseño del conocido JUnit, pero está optimizada para diferentes lenguajes de programación. Por ejemplo, Xunit.net es popular en entornos .NET. Su diseño proporciona una manera estructurada de escribir y organizar pruebas. Además, se caracteriza por soportar atributos para categorizar pruebas, permitir inyecciones de dependencias y mejorar la eficiencia de la ejecución mediante paralelismo.

2.8.6 Apache JMeter

Apache JMeter es una herramienta de pruebas de carga de código abierto diseñada principalmente para aplicaciones web. Es capaz de simular múltiples usuarios concurrentes y evaluar el rendimiento de servidores, aplicaciones y bases de datos. JMeter permite realizar pruebas de carga, pruebas de estrés y análisis de rendimiento, y es compatible con una variedad de protocolos, como HTTP, HTTPS, FTP, y JDBC. Su interfaz gráfica y capacidad de generar informes detallados lo convierten en una herramienta muy útil para proyectos que quieren evaluar la escalabilidad y robustez de sus sistemas.



Apache JMeter

Fuente: <https://github.com/apache/jmeter>

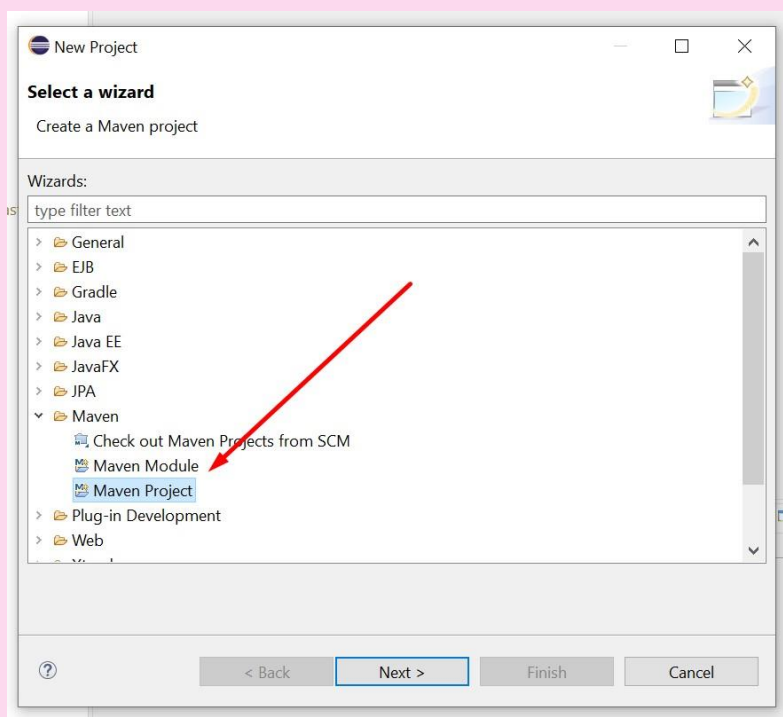


EJEMPLO PRÁCTICO

Hemos comenzado la realización del módulo de asistencia de jugadores mediante Java y Eclipse. Queremos también incorporar Maven y Junit para el desarrollo de nuestras pruebas unitarias, ya que así se especifica y lo realiza el resto de equipo de trabajo.

¿Cuáles serían los pasos?

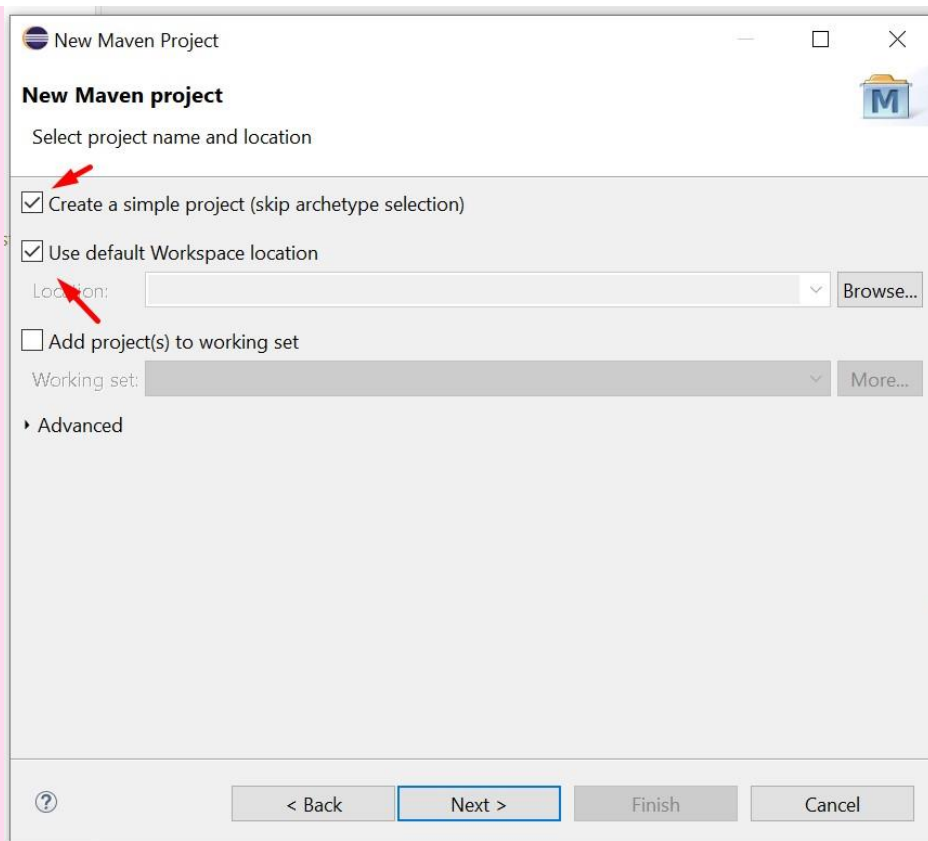
1. El primer paso es crear un nuevo proyecto con Eclipse usando Maven como gestor de librerías y bibliotecas.



Nuevo proyecto con Maven

Fuente: Elaboración Propia

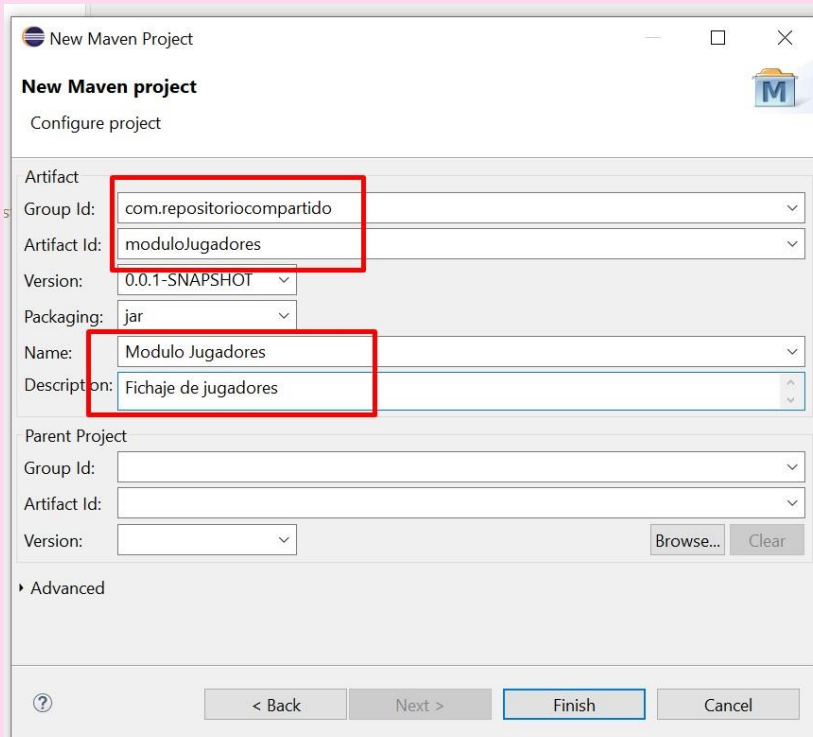
2. Seleccionamos el WorkSpace por defecto y un fichero por defecto de configuración.



Configuración Maven I

Fuente: Elaboración Propia

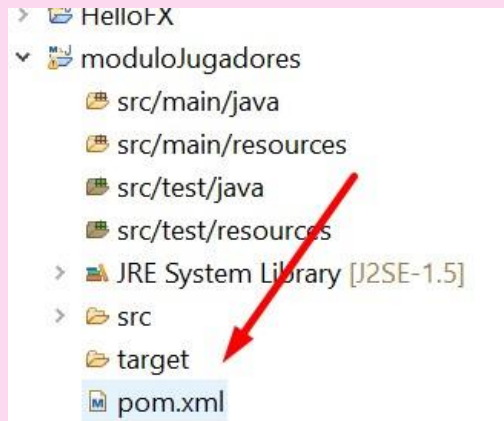
3. Rellenaremos los campos de descripción de nuestro nuevo proyecto.



Configuración Maven II

Fuente: Elaboración Propia

4. Para incorporar las librerías de Junit al proyecto abriremos el fichero de configuración Maven.



Configuración Maven III

Fuente: Elaboración Propia

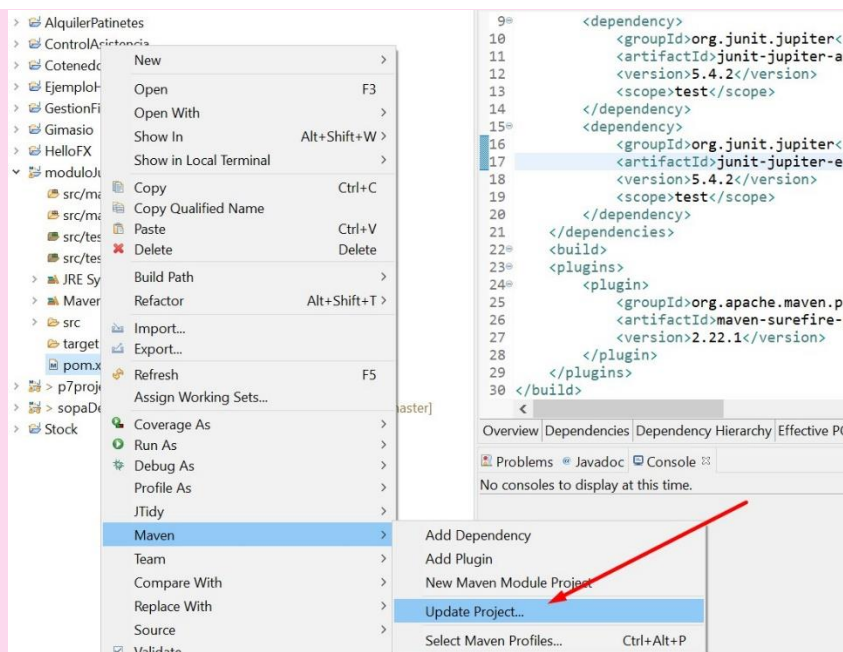
5. Añadimos las dos dependencias necesarias.

```
<dependencies>
  <dependency>
    <groupId>org.junit.jupiter</groupId>
    <artifactId>junit-jupiter-api</artifactId>
    <version>5.4.2</version>
    <scope>test</scope>
  </dependency>
  <dependency>
    <groupId>org.junit.jupiter</groupId>
    <artifactId>junit-jupiter-engine</artifactId>
    <version>5.4.2</version>
    <scope>test</scope>
  </dependency>
</dependencies>
```

6. También añadimos el plugin para la dependencia de compilación.

```
<plugins>
  <plugin>
    <groupId>org.apache.maven.plugins</groupId>
    <artifactId>maven-surefire-plugin</artifactId>
    <version>2.22.1</version>
  </plugin>
</plugins>
```

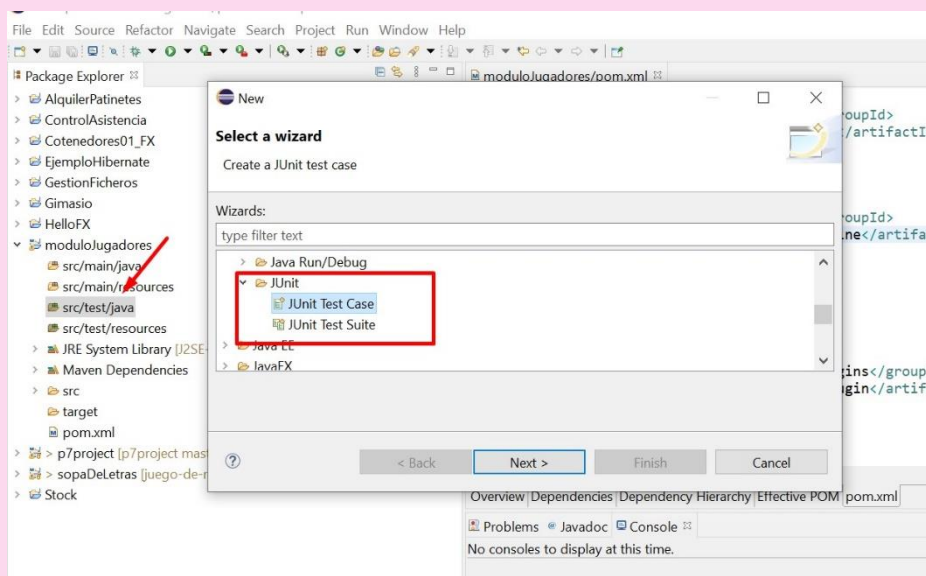
7. Realizamos la actualización de nuestro proyecto.



Actualización del proyecto

Fuente: Elaboración Propia

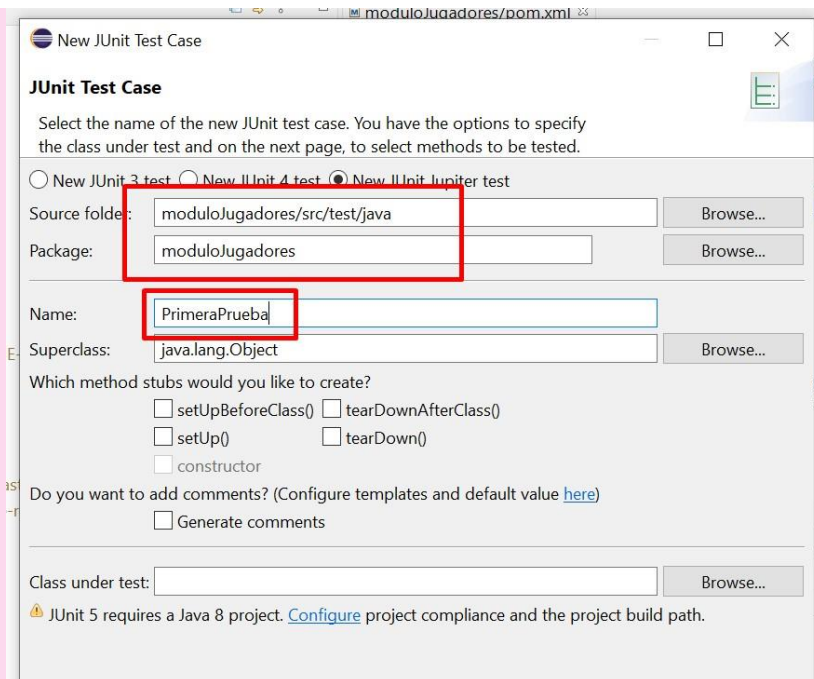
8. Creamos un nuevo test unitario.



Nuevo Test Unitario

Fuente: Elaboración Propia

9. Le damos nombre a la clase.



Nombre les Test

Fuente: Elaboración Propia

10. Como observamos, se creará un código por defecto sobre el cual podremos comenzar a crear nuestros primeros test.

```
package moduloJugadores;

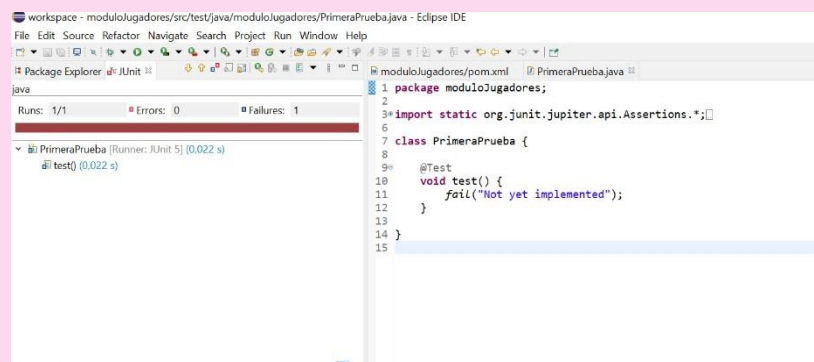
import static org.junit.jupiter.api.Assertions.*;

import org.junit.jupiter.api.Test;

class PrimeraPrueba {

    @Test
    void test() {
        fail("Not yet implemented");
    }

}
```



Nuevo Test Unitario

Fuente: Elaboración Propia

RESUMEN FINAL

Dentro de las fases del desarrollo de aplicaciones, las pruebas son parte esencial para tener una buena calidad de software y además entregar un producto a cliente que cumpla las especificaciones descritas al comienzo del proyecto.

Tal y como hemos visto, la planificación de los objetivos y la temporalización de las diferentes pruebas de las que disponemos es esencial, tan importante incluso como el propio desarrollo del software.

Hemos estudiado que normalmente se planifican primero las pruebas funcionales o unitarias que nos permiten probar de forma individual el correcto funcionamiento de las partes de nuestra aplicación.

A partir de aquí y cuando la aplicación está más desarrollada e incluso finalizada, las pruebas de integración y no funcionales entran a formar parte de nuestra planificación ya que con estas pruebas realizamos comprobaciones del funcionamiento global del software.

Por último, hemos visto como las herramientas, como Junit, nos ayudan a la automatización de los test, y de esa forma también realizar un correcto seguimiento tanto en el momento de la realización de la prueba, como en futuras actualizaciones del software.